Прототип системы автоматического анализа изменений кода

ИТЕРАЦИЯ 2

Команда:

1. Беннер Виктория 3388
2. Титкова Софья 3388
3. Снигирев Александр 3388
4. Лутфулин Даниил 3388
5. Дубровин Данила 3388
6. Чернуха Владимир 1310

План текущей и следующей итерации

План текущей итерации :

Инфраструктура и данные:

- Проектирование и инициализация схемы базы данных.
- Проектирование БД.
- Подготовка инструкции по развертыванию системы.

Backend-разработка:

- **Модуль Gatekeeper:** Прием входящих событий (webhook) и их постановка в очередь брокера.
- **Модуль Consumer:** Обработка задач из очереди, интеграция с LLM и автоматическая публикация результатов в GitHub.

Frontend-разработка:

- Верстка пользовательского интерфейса по утвержденным макетам.

Сборка и запуск:

- Создание Docker-образов для всех компонентов.
- Сборка первого рабочего билда для демонстрации полного цикла работы системы.

План следующей итерации:

Реализовать сбор статистики в анализаторе: Добавить классификацию ревью (approved/needs_changes) с сохранением метаинформации (PR ID, timestamp, модель LLM).

Реализовать отправку статистики: Настроить передачу данных из consumer в dispatcher через брокер или HTTP с сохранением в PostgreSQL.

Реализовать связь dispatcher — frontend: Добавить REST-эндпоинты для статистики и историю ревью; интегрировать фронтенд.

Написать тесты: Добавить unit- и интеграционные тесты с проверкой покрытия в CI (≥80%).

Переписать логирование: Заменить на JSON-логи с уровнями DEBUG/INFO/ERROR для всех сервисов.

Результаты текущей итераций

Результаты текущей итерации — что сделано

Интерфейс: Сверстана страница по макету.

База данных: В Wiki описана схема данных и выполнена инициализация.

LLM: Проведено сравнение LLM.

Документация: Подготовлена инструкция по развертыванию.

GitHub Webhooks: Рассмотрен переход на GitHub Webhooks. Изучена и задокументирована настройка (инструкция). На сервере реализован прием webhook-запросов и отправка в брокер.

Dispatcher: Реализована часть dispatcher: прием запросов от webhook и отправка в брокер.

Consumer: Реализован consumer: получение сообщений из брокера, отправка в LLM, публикация комментария (ревью) в GitHub.

Сверстанная страница

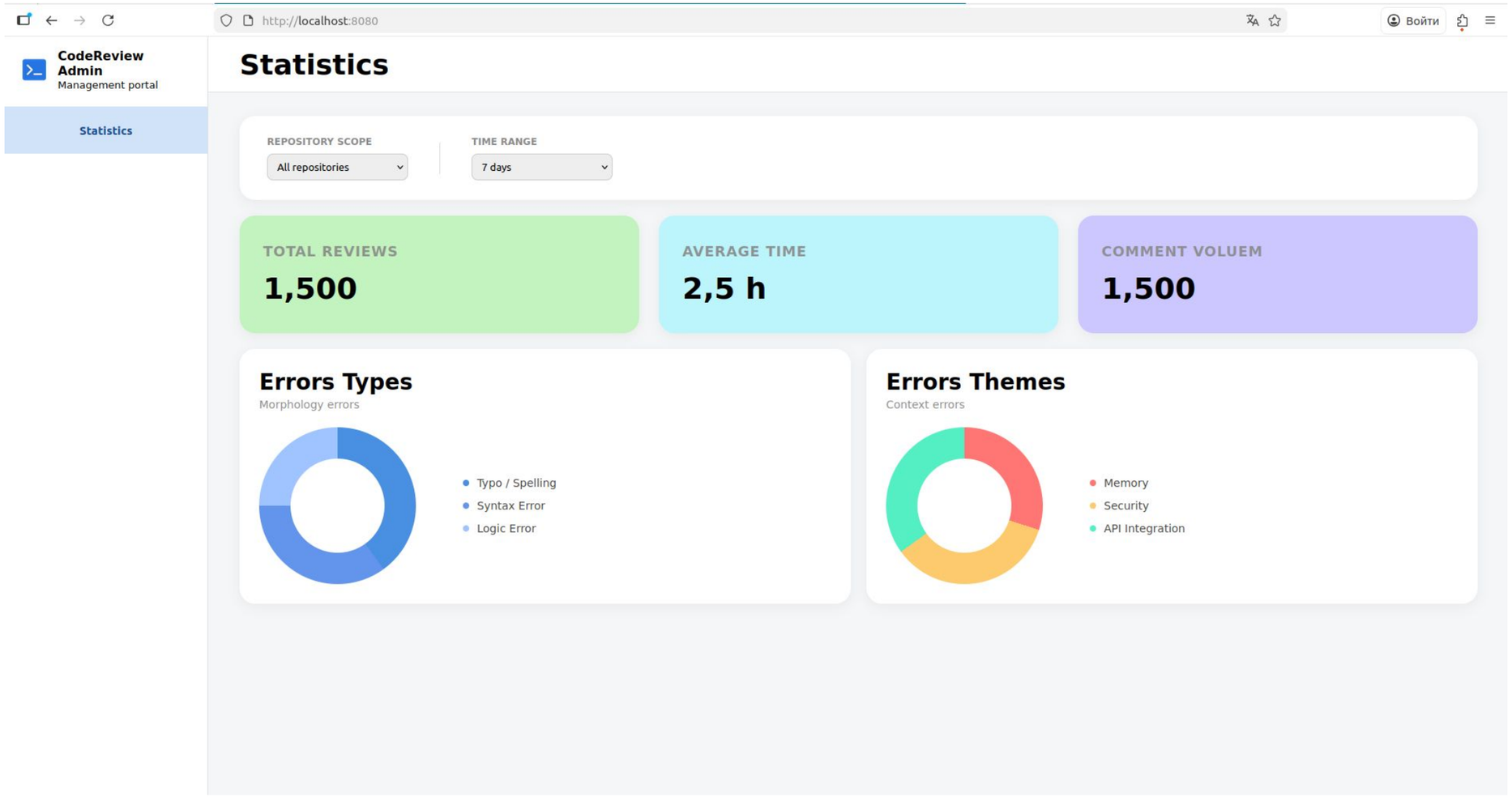


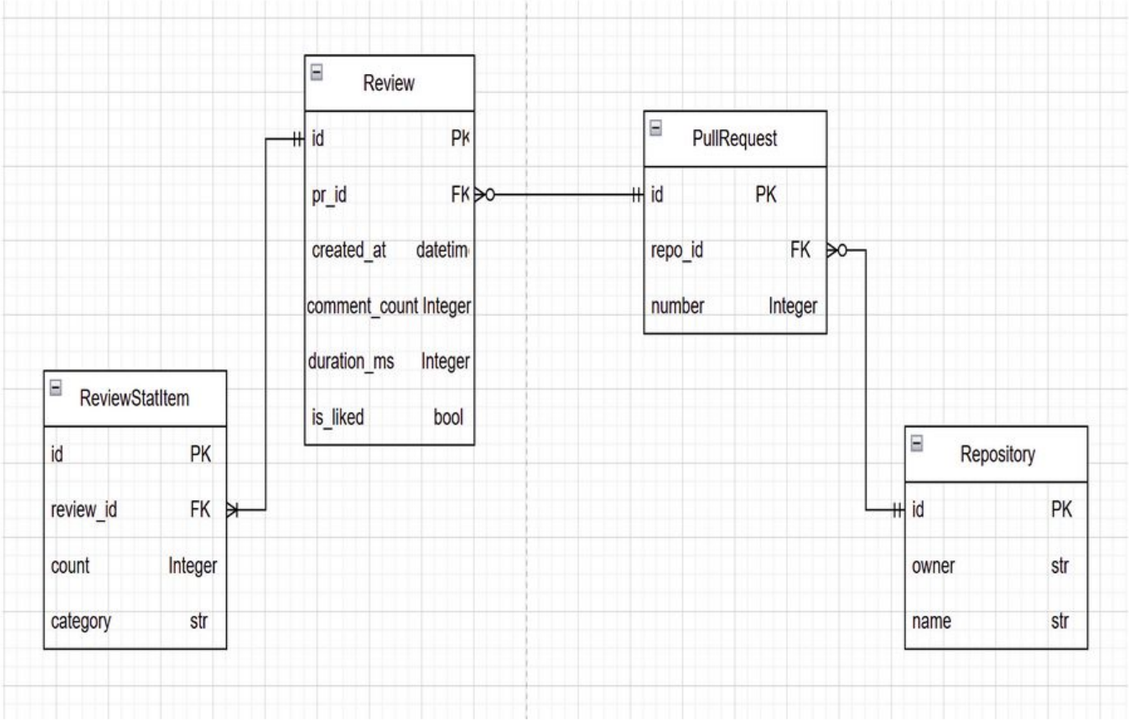
Схема БД и связи

Схема БД

Alexander-Snigirev edited this page 2 weeks ago · 2 revisions

Edit New page

База данных



Связи

Repository - PullRequest: 1-N - У репозитория может быть много ПР, а может не быть ни одного

PullRequest - Review: 1-N - У пулл-реквеста может быть несколько ревью, а может не быть ни одного

Review - ReviewStateItem: 1-N - Ревью обязательно имеет статистику по ошибкам (даже если их количество == 0)

Pages 6

Find a page or section...

Home

Выбор и обоснование LLM

Инструкция по настройке Github...

Информация по проекту

Схема БД

База данных

Связи

Требования и сценарии использ...

+ Add a custom sidebar

Clone this wiki locally

https://github.com/moevm/mse1h202

Wiki страницы

Инструкция по настройке Github Webhooks

Vladimir Chernukha edited this page 5 days ago · 2 revisions

Настройка интеграции через GitHub Webhooks API

Для работы системы необходимо настроить передачу событий из GitHub в ваш сервис `Dispatcher`. Это позволит бэкенду получать уведомления о новых комментариях в режиме реального времени.

1. Обеспечение внешнего доступа

Поскольку GitHub — это облачный сервис, он должен иметь доступ к вашему локальному эндпоинту через интернет:

- **Публичный IP**: Если сервер развернут в дата-центре.
- **Самый простой вариант для тестового использования через smee**: Переходим на сайт https://smee.io/ и жмём кнопку Start a new channel, копируем адрес который вывелся на странице (это наш публичный адрес). Далее вводим в консоли команды:

```
npm install --global smee-client # Это для установки, если smee уже установлен то вводить не надо
smee --url {АДРЕС СКОПИРОВАННЫЙ С САЙТА} --target http://127.0.0.1:8000/worker/webhook # Запускаем утилиту,
```

- **Туннелирование (localtunnel / ngrok)**: Если разработка ведется локально. Запустите туннель на порту `8080`.
- Скопируйте полученный публичный адрес (далее — `BASE_URL`).

2. Регистрация Webhook в GitHub

1. Перейдите в настройки вашего репозитория: **Settings** -> **Webhooks**.
2. Нажмите кнопку **Add webhook**.

3. Конфигурация сетевых параметров

Параметр	Техническое значение	Описание
Payload URL	`${BASE_URL}/webhook` (или просто `{BASE_URL}`	endpoint в сервисе `Dispatcher`

Выбор и обоснование LLM

3388_Беннер_Виктория_Александровна edited this page last week · 1 revision

Edit New page

Выбор и обоснование LLM для Аналитического модуля (Self-hosted)

Сравнительная таблица технологий

Ниже приведено сравнение наиболее эффективных Open-Source моделей для работы с кодом по состоянию на 2024-2025 гг.

Модель	HumanEval (Pass@1)	MBPP (Python)	VRAM (для 4-bit)	Вердикт
DeepSeek-Coder-V2 (Lite)	**81.1%**	81.3%	~12-16 ГБ	**Выбираем для Prod**
Qwen-2.5-Coder-7B	79.9%	**82.6%**	~8-10 ГБ	Сильный аналог для Prod
Llama-3-8B-Instruct	62.2%	67.9%	~8 ГБ	Среднее качество ревью
Qwen-2.5-Coder-1.5B	50.0%	60.0%	**~2-3 ГБ**	**Выбираем для Dev**
CodeLlama-7B	33.5%	52.1%	~8 ГБ	Устарела

Пояснения к критериям:

- **HumanEval / MBPP**: Стандартные тесты на способность модели писать корректный код и решать алгоритмические задачи. Чем выше процент, тем меньше ошибок в советах ИИ.
- **VRAM (видеопамять)**: Ключевой параметр для требования **Self-hosted**. Модели до 7-16 ГБ можно запускать на одной доступной видеокарте.

Pages 6

Find a page or section...

Home
Выбор и обоснование LLM
Выбор и обоснование LLM для Аналитического модуля (Self-hosted)
Сравнительная таблица технологий
Пояснения к критериям:
Анализ:
Итоговый вывод и рекомендации
1. Продакшн-решение: DeepSeek-Coder-V2 (Lite-Instruct)
2. Легкое решение для разработки (Dev): Qwen-2.5-Coder-1.5B
Бенчмарки
Инструкция по настройке Github...
Информация по проекту
Схема БД
Требования и сценарии использ...

+ Add a custom sidebar

Схема БД
Требования и сценарии использ...
+ Add a custom sidebar

Clone this wiki locally

Инструкция по запуску

README

mse-template

Установка и запуск

Перед запуском проекта убедитесь, что у вас установлен Docker

```
docker --version
```

```
docker compose version
```

Если Docker не установлен, скачайте и установите его с официального сайта:

- Windows / macOS: https://www.docker.com/products/docker-desktop/
- Linux: следуйте инструкции для вашего дистрибутива на https://docs.docker.com/engine/install/

1. Клонирование репозитория

```
git clone https://github.com/moevm/mse1h2026-review
```

Перейдите в папку проекта

```
cd mse1h2026-review
```

Альтернативный способ: загрузка архива

Можно просто скачать проект архивом:

1. Перейдите на страницу репозитория: https://github.com/moevm/mse1h2026-review
2. Нажмите кнопку **Code → Download ZIP**
3. Распакуйте скачанный архив в удобное место

README

https://github.com/moevm/mse1h2026-review

2. Нажмите кнопку **Code → Download ZIP**
3. Распакуйте скачанный архив в удобное место
4. Откройте терминал из распакованной папки проекта

2. Настройка окружения

```
cp .env.example .env
```

Откройте файл .env в любом текстовом редакторе и укажите ваши данные:

```
GITHUB_TOKEN=ghp    - ваш токен доступа (Access Token).
GITHUB_USER=User    - ваш логин на GitHub.
```

3. Запуск проекта

```
docker compose up -d
```

4. Проверка статуса

```
docker compose ps
```

Все сервисы должны быть в статусе Up или healthy.

5. Доступ к проекту

После успешного запуска проект будет доступен по адресу:

```
http://localhost:8080
```

6. Остановка проекта

```
docker compose down
```